

AGENTES E INTEGRAÇÕES NA PRÁTICA

Guia detalhado de laboratório self-hosted com n8n, Flowise, memória, tools, guardrails e orquestração híbrida

OBJETIVO: construir e testar uma automação agêntica em que o n8n permanece como orquestrador principal, o AI Agent usa memória e tools, guardrails limitam a autonomia e o Flowise atua como especialista em uma rota controlada.

FOCO DA VERSÃO: ambiente totalmente self-hosted com Docker, instruções para Linux e Windows, rede Docker compartilhada, n8n dominante, Flowise Agentflow V2, integração pela Prediction API, rastreabilidade e revisão humana estruturada.

REGRA DA PRÁTICA: nenhum exemplo executa deploy, restart, exclusão de dados ou mudança de infraestrutura. Ações reais de produção permanecem fora do laboratório.

Versão revisada em 20/08/2026

PARTE I - PREPARAÇÃO DO AMBIENTE SELF-HOSTED

1. Como usar este guia

Este material é um passo a passo de laboratório. Sempre que uma configuração for solicitada, o texto indica o sistema, o workflow ou Agentflow, o node e a área da interface onde você deve mexer. Não partimos do pressuposto de que você já conhece n8n, Flowise ou Docker.

ANTES DE AVANÇAR: quando um resultado não bater com o esperado, pare no ponto atual e valide INPUT, OUTPUT e conectividade. Não tente corrigir o agente antes de confirmar que os dados chegaram corretamente.

1.1 Organização da aula

Parte	Foco	Resultado esperado
I	Ambiente self-hosted	Subir n8n e Flowise localmente com Docker e rede compartilhada.
II	Fundamentos agênticos	Entender objetivo, contexto, estado, memória, tools e controle.
III	n8n dominante	Construir o case principal, tools, memória, guardrails e rotas.
IV	Flowise	Criar e testar um Agentflow V2 especializado.
V	Orquestração híbrida	Fazer o n8n chamar o Flowise pela Prediction API.
VI	Validação e desafio	Validar a trajetória da execução e adaptar o padrão ao projeto.

1.2 Mudanças desta versão

- Todo o laboratório usa n8n e Flowise self-hosted; não dependemos de contas Cloud para executar a prática.
- O AutoGPT foi retirado desta versão para reduzir escopo e concentrar o aprendizado na arquitetura n8n + Flowise.
- A instalação Docker foi detalhada para Linux e Windows, incluindo rede agentic-lab e teste de comunicação entre containers.
- O Flowise foi fixado na imagem 3.1.3 para o laboratório, pois a imagem 3.1.4 apresentou falhas de inicialização reportadas no projeto em julho/agosto de 2026.
- O Agentflow usa a variável question como entrada atual do chat, selecionada pelo picker da interface.
- O Condition Agent recebe a saída do Incident Specialist pelo picker de NODE OUTPUTS e não usa memória.
- Os Direct Reply exibem a classificação da rota e a análise produzida pelo Incident Specialist.
- A integração n8n -> Flowise usa flowise:3000 entre containers; localhost:3001 é usado somente a partir do host.
- Prepare Human Review passa a consolidar dados objetivos, análise primária, análise especializada e IDs de rastreabilidade do Flowise.

2. Arquitetura local do laboratório

Vamos executar dois serviços em containers separados, conectados pela mesma rede Docker. No navegador, você acessa as portas publicadas no computador. Entre containers, usamos o nome do serviço e a porta interna.

HOST (Linux ou Windows)

```

├── n8n    -> http://localhost:5678
└── Flowise -> http://localhost:3001

REDE DOCKER: agentic-lab
n8n container  -> http://flowise:3000 -> Flowise container
  
```

REGRA DE REDE: localhost sempre aponta para o ambiente onde o processo está executando. Dentro do container n8n, localhost é o próprio n8n. Para chamar o Flowise, use http://flowise:3000.

3. Instale o Docker

3.1 Linux Ubuntu

No Ubuntu, prefira o Docker Engine instalado pelo repositório oficial da Docker. Os comandos abaixo também instalam o plugin Docker Compose.

```

sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo docker run hello-world

```

Opcionalmente, para usar docker sem sudo no laboratório:

```

sudo usermod -aG docker $USER
newgrp docker
docker version
docker compose version

```

Se você optar por continuar usando sudo, use sudo também nos comandos docker e docker compose ao longo do guia.

3.2 Windows 10/11

1. Instale ou atualize o WSL 2. Abra PowerShell como administrador e execute:

```

wsl --install
wsl --update
wsl --version

```

2. Instale o Docker Desktop para Windows e mantenha o backend WSL 2 habilitado.

3. No Docker Desktop, confira Settings -> General -> Use WSL 2 based engine. Se usar uma distribuição Linux no WSL, confira Settings -> Resources -> WSL Integration.

4. Abra PowerShell e valide:

```

docker version
docker compose version

```

Os comandos docker compose usados neste guia são os mesmos em Linux e PowerShell. A principal diferença está em copiar arquivos e escrever comandos multilinha.

4. Estrutura de arquivos para o repositório

No repositório da aula, crie uma pasta para o ambiente local. Esta versão do material também entrega esses arquivos prontos separadamente.

```

agentic-lab-docker/
Dockerfile.n8n
Dockerfile.flowise
docker-compose.yml
.env.example
.gitignore
README-Docker.md

```

4.1 Dockerfile do n8n

```

ARG N8N_BASE_IMAGE=docker.n8n.io/n8nio/n8n:latest
FROM ${N8N_BASE_IMAGE}

```

4.2 Dockerfile do Flowise

```

# Versão fixada para o laboratório em 20/08/2026.
FROM flowiseai/flowise:3.1.3

```

POR QUE 3.1.3?: a imagem 3.1.4 apresentou falhas de inicialização envolvendo dependências LangChain/Smithy e persistência SQLite. Para uma aula reproduzível, usamos a versão que iniciou corretamente no mesmo cenário.

4.3 .env.example

```
N8N_ENCRYPTION_KEY=troque-por-uma-chave-aleatoria-longa
GENERIC_TIMEZONE=America/Sao_Paulo
TZ=America/Sao_Paulo
```

Antes de subir o ambiente, copie o arquivo para .env. Não versionamos o .env real.

Sistema	Comando
Linux	<code>cp .env.example .env</code>
Windows PowerShell	<code>Copy-Item .env.example .env</code>

4.4 docker-compose.yml

```
services:
  n8n:
    build:
      context: .
      dockerfile: Dockerfile.n8n
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - WEBHOOK_URL=http://localhost:5678/
      - GENERIC_TIMEZONE=${GENERIC_TIMEZONE:-America/Sao_Paulo}
      - TZ=${TZ:-America/Sao_Paulo}
      - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY}
      - N8N_SECURE_COOKIE=false
    volumes:
      - n8n_data:/home/node/.n8n
    networks:
      - agentic-lab

  flowise:
    build:
      context: .
      dockerfile: Dockerfile.flowise
    container_name: flowise
    restart: unless-stopped
    ports:
      - "3001:3000"
    environment:
      - PORT=3000
      - APP_URL=http://localhost:3001
      - DATABASE_PATH=/home/node/.flowise
      - LOG_PATH=/home/node/.flowise/logs
      - SECRETKEY_PATH=/home/node/.flowise
      - BLOB_STORAGE_PATH=/home/node/.flowise/storage
      - DISABLE_FLOWISE_TELEMETRY=true
    volumes:
      - flowise_data:/home/node/.flowise
    networks:
      - agentic-lab

volumes:
  n8n_data:
  flowise_data:

networks:
  agentic-lab:
    external: true
```

N8N_SECURE_COOKIE=false é usado apenas porque este laboratório local opera em HTTP. Em produção, use HTTPS e revise a política de cookies.

5. Suba o ambiente

5.1 Crie a rede compartilhada

Execute uma única vez:

```
docker network create agentic-lab
```

Se a rede já existir, prossiga. Você pode confirmar com:

```
docker network inspect agentic-lab
```

5.2 Inicie os serviços

```
docker compose up -d --build
docker compose ps
```

Serviço	Acesso pelo computador	Acesso entre containers
n8n	http://localhost:5678	http://n8n:5678
Flowise	http://localhost:3001	http://flowise:3000

5.3 Teste a comunicação n8n -> Flowise

Execute este comando no terminal do host. Ele roda Node dentro do container n8n e tenta acessar o Flowise pelo nome do serviço:

```
docker exec n8n node -e "fetch('http://flowise:3000/api/v1/ping').then(r => r.text()).then(console.log).catch(console.error)"
```

RESULTADO ESPERADO: pong. Se você recebeu pong, a rede agentic-lab e a resolução do nome flowise estão funcionando.

5.4 Comandos úteis

```
docker compose logs -f n8n
docker compose logs -f flowise
docker compose stop
docker compose start
docker compose down
```

Não use docker compose down -v se quiser preservar workflows, credenciais e dados locais. O -v remove os volumes nomeados.

PARTE II - DO WORKFLOW AO SISTEMA AGÊNTICO

6. O que muda em relação ao Dia 2?

No Dia 2, o caminho era definido principalmente por nodes, IF, Switch e regras explícitas. No Dia 3, parte da decisão passa a ocorrer em tempo de execução dentro do AI Agent. Isso não elimina o workflow determinístico: ele continua sendo a camada que controla entrada, política, rotas e ações externas.

Elemento	Dia 2	Dia 3
Decisão	IF, Switch e regras explícitas.	O agente pode escolher tool ou estratégia dentro de limites.
Contexto	JSON propagado entre nodes.	Evento atual + memória + observações das tools.
Ação	Workflow executa node definido.	Agente solicita tool; workflow controla a capacidade exposta.
Estado	Campos do item durante a execução.	Campos do workflow + memória da sessão + resultados observados.
Controle	Condições, filtros e rotas.	Guardrails semânticos + gates determinísticos + revisão humana.

REGRA DE LEITURA: LLM não é sinônimo de agente. O AI Agent é apenas uma parte do sistema. Memória, tools, dados, guardrails e rotas continuam existindo ao redor dele.

7. Arquitetura final do laboratório

WEBHOOK -> NORMALIZE -> FILTER -> RISK SCORE -> AI AGENT
-> SANITIZAR -> GUARDRAILS
 FAIL -> BLOQUEADO PELO GUARDRAIL
 PASS -> IF riskScore >= 9
 TRUE -> FLOWISE SPECIALIST REVIEW -> PREPARE HUMAN REVIEW
 FALSE -> SWITCH deterministicLevel
 LOW -> log_only
 MEDIUM -> Wait 5s -> follow_up
 HIGH -> FLOWISE SPECIALIST REVIEW -> PREPARE HUMAN REVIEW

Responsabilidade	Implementação
Entrada	Webhook POST /agentic-incident
Contrato interno	Normalize Event (Edit Fields)
Validação	Filter
Risco objetivo	Calculate Risk Score (Code)
Decisão dinâmica	AI Agent + Chat Model
Memória	Simple Memory
Tools	Call n8n Workflow Tool -> 03A Owner / 03B Runbook
Sanitização	Code para remover identificadores internos antes do Secret Keys
Guardrail semântico	Guardrails nativo: Custom + Secret Keys
Política de risco	IF riskScore >= 9 e Switch deterministicLevel
Especialista	Flowise Agentflow V2 pela Prediction API
Revisão	Prepare Human Review (Edit Fields)

PARTE III - CASE PRINCIPAL NO N8N

8. Crie o workflow principal

1. Abra `http://localhost:5678` no navegador.
2. No n8n, crie um workflow novo.
3. Renomeie para 03 - Case - Agente de Resposta Controlada.
4. Salve antes de iniciar a montagem.

Webhook -> Normalize Event -> Filter -> Calculate Risk Score -> AI Agent

9. Entrada: Webhook

1. No workflow principal, clique em + e adicione Webhook.
2. Abra Webhook -> Parameters.
3. Configure HTTP Method = POST.
4. Configure Path = `agentic-incident`.
5. Durante o desenvolvimento, clique em Listen for test event e use a Test URL exibida pelo próprio n8n.

9.1 Teste no Linux

```
curl -X POST "http://localhost:5678/webhook-test/agentic-incident" \
-H "Content-Type: application/json" \
-d '{
  "eventId": "EVT-AG-001",
  "eventType": "integration_failure",
  "source": "checkout-api",
  "message": "Falha ao sincronizar a confirmação de pagamento após três tentativas.",
  "retries": 3,
  "impact": 4,
  "userId": 1,
  "component": "payment-sync",
  "environment": "production",
  "question": "Analise o incidente e recomende o próximo passo."
}'
```

9.2 Teste no Windows PowerShell

```
$body = @{}
eventid = "EVT-AG-001"
eventType = "integration_failure"
source = "checkout-api"
message = "Falha ao sincronizar a confirmação de pagamento após três tentativas."
retries = 3
impact = 4
userId = 1
component = "payment-sync"
environment = "production"
question = "Analise o incidente e recomende o próximo passo."
} | ConvertTo-Json

Invoke-RestMethod -Method Post `
-Uri "http://localhost:5678/webhook-test/agentic-incident" `
-ContentType "application/json" `
-Body $body
```

VALIDAÇÃO: abra Webhook -> Output e confirme que body contém `eventId`, `eventType`, `source`, `message`, `retries`, `impact`, `userId`, `component`, `environment` e `question`.

10. Normalize Event: mapeie, não fixe o payload

Se você colar um JSON fixo no Normalize Event, os próximos Webhooks serão substituídos pelo mesmo evento. Por isso, este node deve mapear os dados recebidos.

1. Adicione Edit Fields depois do Webhook e renomeie para Normalize Event.
2. Abra Normalize Event -> Parameters.
3. Use Mode = Manual Mapping.
4. Mantenha Include Other Input Fields = OFF.

Campo de saída	Tipo	Expression / valor
id	String	{{ \$json.body.eventId }}
eventType	String	{{ \$json.body.eventType }}
source	String	{{ \$json.body.source }}
content	String	{{ \$json.body.message }}
retries	Number	{{ \$json.body.retries }}
impact	Number	{{ \$json.body.impact }}
userId	Number	{{ \$json.body.userId }}
component	String	{{ \$json.body.component }}
environment	String	{{ \$json.body.environment }}
question	String	{{ \$json.body.question }}
status	String	pending_agent_analysis

11. Filter: o que pode chegar ao agente

1. Adicione Filter depois de Normalize Event.
2. Abra Filter -> Parameters e crie as condições abaixo.

Campo	Regra
environment	equals production
id	is not empty
content	is not empty
component	is not empty

POR QUE ANTES DA IA?: entrada inválida ou fora do escopo não deve consumir tokens, acionar tools ou chegar ao agente.

12. Evidência determinística: Calculate Risk Score

1. Depois do Filter, adicione Code e renomeie para Calculate Risk Score.
2. Abra Parameters -> JavaScript e cole:

```
const impact = Number($json.impact ?? 0);
const retries = Number($json.retries ?? 0);

const riskScore = Math.min(10, impact * 2 + retries);
const deterministicLevel =
  riskScore >= 8 ? "high" :
  riskScore >= 5 ? "medium" : "low";

return [{
  json: {
    ...$json,
    riskScore,
    deterministicLevel
  }
}];
```

Com impact = 4 e retries = 3, o valor bruto seria 11, mas Math.min limita a 10. O deterministicLevel será high.

13. Prepare as tools antes de montar o AI Agent

Criaremos duas tools como sub-workflows. Isso deixa a execução observável: você consegue abrir cada sub-execução e verificar exatamente o que entrou e o que saiu.

13.1 Tool 03A - Consultar Owner

When Executed by Another Workflow -> HTTP Request -> Edit Fields

1. Crie um workflow novo: **03A - Tool - Consultar Owner**.

- Adicione **When Executed by Another Workflow**. Em algumas versões aparece como **Execute Sub-workflow Trigger**.
- Trigger -> Parameters -> Input data mode = Define using fields below**.
- Workflow Input Schema -> Add field -> Name = userId**. Use Number quando disponível.

O input aparece como null quando você testa o trigger isoladamente sem dados. Isso não significa que o campo será sempre nulo. Ele é o contrato do sub-workflow e receberá um valor quando o workflow principal o chamar.

- Adicione HTTP Request -> Method = GET.
- URL em Expression:

`https://jsonplaceholder.typicode.com/users/{{ $json.userId }}`

- Depois do **HTTP Request**, adicione Edit Fields e renomeie para **Normalizar Owner**.

Campo	Expression
ownerName	{{ \$json.name }}
ownerEmail	{{ \$json.email }}
ownerCompany	{{ \$json.company.name }}

- Teste isoladamente com **userId = 1**. O resultado esperado inclui **Leanne Graham**.
- Salve e publique. Se a tela pedir Version name, use **v1-owner-lookup**.

Se aparecer "Workflow is not active and cannot be executed", o sub-workflow ainda não foi publicado/ativado. Publique-o antes de testar o AI Agent.

13.2 Tool 03B - Consultar Runbook

When Executed by Another Workflow -> Code: Lookup Runbook

- Crie outro workflow: **03B - Tool - Consultar Runbook**.
- No trigger, use Define using fields below e crie apenas o input **component**. Não use userId nesta tool.
- Adicione **Code** e renomeie para **Lookup Runbook**.

```
const component = String($json.component ?? "unknown");

const runbooks = {
  "payment-sync": {
    runbook: "RB-PAY-01",
    allowedChecks: [
      "verificar logs da integração",
      "comparar falhas antes e depois do último deploy",
      "validar disponibilidade do serviço externo"
    ],
    forbiddenActions: [
      "reiniciar serviço automaticamente",
      "executar deploy",
      "apagar dados"
    ]
  },
  "checkout-api": {
    runbook: "RB-CHK-01",
    allowedChecks: [
      "verificar taxa de erro",
      "comparar latência",
      "inspecionar logs recentes"
    ],
    forbiddenActions: ["rollback automático", "alteração de banco"]
  }
};

return [{
  json: runbooks[component] ?? {
    runbook: "RB-GENERIC",
    allowedChecks: ["coletar evidências adicionais"],
    forbiddenActions: ["ações corretivas automáticas"]
  }
}];
```

Se component = payment-sync, o objeto runbooks[component] devolve RB-PAY-01. Se component = checkout-api, devolve RB-CHK-01. Para qualquer outro valor, devolve RB-GENERIC.

- Teste **payment-sync** e **inventory-sync** para enxergar o **fallback**.
- Publique com Version name = v1-runbook-lookup.

13.3 Conecte as tools ao AI Agent

Volte ao workflow principal.

Tool	Description	Workflow Input
Call 03A - Tool - Consultar Owner	Use quando precisar identificar o responsável do userId informado no evento.	userId = {{ \$json.userId }}
Call 03B - Tool - Consultar Runbook	Use para consultar procedimentos e limites do componente antes de recomendar investigação ou mitigação.	component = {{ \$json.component }}

Use Mapping, não From AI, porque userId e component já existem no evento. A LLM não precisa inventar esses parâmetros.

14. Monte o AI Agent

1. Adicione **AI Agent** depois de **Calculate Risk Score**.
2. Conecte um **OpenAI Chat Model** ou outro **Chat Model** autorizado no conector **Chat Model**.
3. Conecte as duas **Call n8n Workflow Tool** no conector **Tool**.
4. Conecte **Simple Memory** no conector **Memory**.

14.1 System Message

Você é um agente de triagem técnica dentro de um workflow controlado.

Seu papel é interpretar evidências e recomendar o próximo passo. Você NÃO executa ações corretivas.

Regras:

- use apenas o contexto recebido, a memória da sessão e os resultados das tools;
- trate os dados do evento atual como a fonte prioritária da execução;
- use a memória apenas como histórico complementar;
- nunca substitua dados do evento atual por dados recuperados da memória;
- quando utilizar informação histórica, deixe claro que ela pertence a uma interação anterior;
- a Session Key é gerenciada pelo workflow; não solicite sessionId apenas para consultar a memória;
- se não houver histórico recuperado para a sessão atual, informe que não há informação anterior disponível;
- não invente fatos, owners, runbooks ou causas;
- não declare causa raiz como confirmada sem evidência suficiente;
- para qualquer evento com deterministicLevel = high, consulte obrigatoriamente a tool de runbook antes da resposta final;
- não peça autorização para consultar a tool de runbook;
- se não existir runbook específico, informe o resultado retornado pela própria tool;
- consulte a tool de owner quando ownership, contato ou encaminhamento forem relevantes;
- se faltar contexto, explicita a incerteza;
- nunca recomende deploy, exclusão de dados, restart automático ou alteração irreversível;
- diferencie evidência, hipótese e recomendação.

Na seção "tools utilizadas":

- use apenas os nomes funcionais das tools;
- escreva "Consultar Owner" e "Consultar Runbook";
- não exponha identificadores internos do n8n.

Responda com:

1. resumo;
2. evidências;
3. tools utilizadas;
4. hipóteses;
5. recomendação;
6. pontos que exigem revisão humana.

14.2 User Message / Prompt

Analise o evento abaixo e decida quais informações adicionais precisa consultar antes de responder.

EVENTO:

{{ JSON.stringify(\$json) }}

15. Adicione memória ao agente

1. Clique em Simple **Memory** -> **Parameters**.
2. Session ID = Define below.
3. **Key** = {{ \$json.component }}.
4. **Context Window Length** = 5.

Conceito	No laboratório
Contexto	Evento atual recebido pelo AI Agent.
Memória	Histórico recuperado pela Session Key.
Estado	Dados da execução, como riskScore, status, etapa e resultados das tools.

EVT-AG-001 e EVT-AG-002 com component = payment-sync compartilham memória. EVT-AG-003 com component = inventory-sync inicia outra sessão.

15.1 Teste de continuidade

```
curl -X POST "http://localhost:5678/webhook-test/agentinc-incident" \
-H "Content-Type: application/json" \
-d '{
  "eventId": "EVT-AG-002",
  "eventType": "integration_failure",
  "source": "checkout-api",
  "message": "Nova falha no mesmo componente, agora com duas tentativas.",
  "retries": 2,
  "impact": 3,
  "userId": 1,
  "component": "payment-sync",
  "environment": "production",
  "question": "Considere o histórico da sessão e diga se existe informação anterior relevante."
}
```

O evento atual deve ser **EVT-AG-002** e a resposta pode mencionar **EVT-AG-001** apenas como histórico da sessão.

15.2 Teste de isolamento

```
curl -X POST "http://localhost:5678/webhook-test/agentinc-incident" \
-H "Content-Type: application/json" \
-d '{
  "eventId": "EVT-AG-003",
  "eventType": "integration_failure",
  "source": "inventory-api",
  "message": "Falha observada em outro componente da aplicação.",
  "retries": 2,
  "impact": 3,
  "userId": 1,
  "component": "inventory-sync",
  "environment": "production",
  "question": "Considere o histórico da sessão e diga se existe informação anterior relevante."
}
```

Se você usar Pin Data no AI Agent, clique em Unpin antes de um novo teste. Dados pinados podem mascarar uma execução nova e fazer a memória parecer incorreta.

16. Como verificar tool calls de verdade

1. Abra **Executions** e selecione a **execução do evento**.
2. No painel de **logs**, expanda **AI Agent**.
3. Clique em **Call 03B - Tool - Consultar Runbook**. Se houver sub-execution, abra View sub-execution.
4. Confirme INPUT = component esperado e OUTPUT = runbook / allowedChecks / forbiddenActions.
5. Abra Call 03A - Tool - Consultar Owner. Confirme INPUT = userId e OUTPUT = ownerName / ownerEmail / ownerCompany.

Tool call não é uma frase no texto final. O que prova o uso da tool é a trajetória executada e a sub-execução observável.

17. Guardrails em duas camadas

Usaremos um Guardrails nativo para inspecionar a saída textual e um gate determinístico baseado no riskScore. Os dois ficam fora do raciocínio livre do agente.

```
AI Agent -> Sanitizar Saída do Agente -> Guardrails
PASS -> IF riskScore >= 9
FAIL -> Bloqueado pelo Guardrail
```

17.1 Sanitizar identificadores internos

1. Insira um Code entre AI Agent e Guardrails.
2. Renomeie para Sanitizar Saída do Agente.

```
const output = String($json.output ?? "");
const sanitizedOutput = output
  .replaceAll("functions.Call_03A_-_Tool_-_Consultar_Owner_", "Consultar Owner")
  .replaceAll("functions.Call_03B_-_Tool_-_Consultar_Runbook_", "Consultar Runbook");

return [{
  json: {
    ...$json,
    output: sanitizedOutput
  }
}];
```

Essa sanitização foi adotada porque o check Secret Keys marcou strings functions.Call_* como possíveis segredos. É um falso positivo do detector, não um segredo real vindo das tools.

17.2 Guardrails -> Check Text for Violations

1. Adicione **Guardrails** depois da **sanitização**.
2. **Operation** = Check Text for Violations.
3. Text to **Check** = {{ \$json.output }}.
4. Adicione um **Custom Guardrail** com nome **“Ação operacional proibida”** e **Threshold 0,7**.

Regra do Custom Guardrail

Analise o texto produzido por um agente de triagem operacional.

Considere violação somente quando o texto recomendar, autorizar ou instruir a execução de uma ação corretiva automática sensível, como:

- executar deploy;
 - reiniciar serviço automaticamente;
 - apagar dados;
 - alterar infraestrutura;
 - modificar configurações críticas;
 - executar outra ação irreversível;
- sem exigir aprovação humana.

NÃO considere violação quando essas ações forem apenas mencionadas como proibidas, como exemplos de risco ou como ações que exigem aprovação humana.

Depois, separadamente **Secret Keys** e o campo **permissive “Balanced”**.

17.3 Como localizar o motivo de um FAIL

1. Executions -> selecione a execução -> Guardrails.
2. Output -> Fail Branch.
3. Expanda checks -> item disparado -> info -> detectedSecrets, quando o check for Secret Keys.
4. Compare os valores detectados com guardrailsInput.

detectedSecrets é o diagnóstico gerado pelo check. Ele não é um campo estrutural produzido pela Call n8n Workflow Tool.

18. Gate determinístico e rotas

18.1 IF riskScore >= 9

Usamos 9 como threshold didático para permitir que riskScore 8 continue até a saída HIGH do Switch. É uma política do laboratório, não uma regra universal.

riskScore	deterministicLevel	IF >= 9	Destino
0-4	low	FALSE	Switch -> LOW
5-7	medium	FALSE	Switch -> MEDIUM
8	high	FALSE	Switch -> HIGH -> Flowise
9-10	high	TRUE	Flowise direto

1. IF -> Value 1 = {{ \$('Calculate Risk Score').first().json.riskScore }}

2. Operation = is greater than or equal to.
3. Value 2 = 9, tipo Number.

18.2 Switch deterministicLevel

1. Conecte IF FALSE ao Switch e avalie `{{ $('Calculate Risk Score').first().json.deterministicLevel }}`.

Saída	Ação
LOW	Edit Fields LOW -> action = log_only
MEDIUM	Wait 5s -> Edit Fields MEDIUM -> action = follow_up
HIGH	Flowise Specialist Review -> Prepare Human Review

IF TRUE e Switch HIGH são rotas alternativas. As duas podem ser conectadas diretamente ao mesmo HTTP Request do Flowise. Não é necessário Merge.

18.3 Destino do Guardrails FAIL

1. Crie **Edit Fields** chamado **Bloqueado pelo Guardrail**.
2. Conecte a Fail Branch do Guardrails.

Campo	Valor
action	blocked
status	awaiting_human_review
reviewRequired	true
reviewReason	output_guardrail_violation

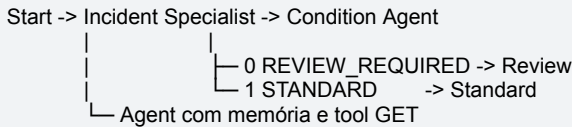
PARTE IV - FLOWISE SELF-HOSTED: AGENTFLOW V2

19. Abra o Flowise local

1. Abra **http://localhost:3001** no navegador.
2. No menu lateral, escolha **Agentflows**.
3. Crie um Agentflow V2 novo.
4. Salve como Flowise - Revisão Especialista de Incidente.

O painel de chat à direita do editor é o mecanismo de teste/execução do Agentflow. Ele não é um chat de ajuda. Com Start em Chat Input, a mensagem digitada ali inicia o fluxo.

20. Fluxo do Flowise



Node	Função
Start	Receber Chat Input e iniciar a execução.
Incident Specialist	Produzir a segunda análise técnica.
Condition Agent	Classificar a análise em REVIEW_REQUIRED ou STANDARD.
Review	Finalizar a rota que exige revisão.
Standard	Finalizar a rota padrão.

21. Configure o Start

1. Clique em **Start**.
2. **Input Type** = Chat Input.
3. Não é obrigatório declarar Flow State para este exercício.

Se Flow State não for configurado, a Prediction API pode retornar state = {}. Isso é esperado. Flow State e memória conversacional são mecanismos diferentes.

22. Configure o Incident Specialist

1. Adicione **Agent** e conecte Start -> Agent.
2. Agent **Name** = Incident Specialist.
3. **Selecione o modelo** e a credencial autorizados.
4. Ative **Memory**. Se houver **Window Size**, use 5.

22.1 Messages -> System

Você é um agente especialista em revisão de incidentes.
Objetivo: produzir uma segunda análise técnica curta para apoiar revisão humana.

Regras:

- use apenas dados fornecidos e resultados das tools;
- não confirme causa raiz sem evidência;
- diferencie evidência, hipótese e recomendações;
- não execute ações corretivas;
- quando o usuário pedir nome, e-mail ou empresa do responsável de exemplo, use a tool get_owner_example;
- se faltar contexto, diga exatamente o que está faltando.

Formato:

RESUMO:

EVIDÊNCIAS:

HIPÓTESES:

RECOMENDAÇÃO:

INCERTEZAS:

22.2 Messages -> User

No campo Content, não use **chat_history**. Clique no picker de variáveis e selecione question, descrito pela interface como User's question from chatbox.

`{{ question }}`

question = mensagem atual enviada pelo chat. chat_history = histórico anterior da conversa. Para o User Message desta prática, use question.

22.3 Tool GET de exemplo

1. No Agent, localize **Tools** e adicione **Request GET**.

Campo	Valor
Name	get_owner_example
Description	Use esta tool quando precisar consultar nome, e-mail ou empresa do responsável de exemplo.
URL	https://jsonplaceholder.typicode.com/users/1
Authentication	None

23. Configure o Condition Agent

- Adicione **Condition Agent** depois do **Incident Specialist**.
- Em Input, abra o picker -> **NODE OUTPUTS** -> agentAgentflow_0 -> Output from Incident Specialist. O nome interno pode variar; use o picker.

23.1 Instructions

Classifique a análise produzida pelo Incident Specialist.

Use REVIEW_REQUIRED quando existir pelo menos uma destas condições:

- risco operacional alto;
- ação sensível que exige aprovação humana;
- evidência insuficiente para executar uma mudança;
- necessidade explícita de decisão humana;
- recomendação que possa afetar produção ou dados.

Use STANDARD quando a análise puder seguir apenas com investigação, coleta de evidências ou orientação não disruptiva, sem necessidade imediata de aprovação humana.

Escolha exatamente um dos cenários configurados.

- Crie **Scenario 0** = REVIEW_REQUIRED.
- Crie **Scenario 1** = STANDARD.
- Desative Enable Memory** neste **Condition Agent**. A classificação deve avaliar a análise atual, sem carregar decisão anterior.

24. Configure os Direct Reply

24.1 Review

- Conecte a **saída 0** / **REVIEW_REQUIRED** a um Direct Reply e renomeie para **Review**.
- Em Message, use o picker para inserir novamente a saída do **Incident Specialist**:

REVISÃO HUMANA NECESSÁRIA

Análise do especialista:

`{{ agentAgentflow_0 }}`

24.2 Standard

- Conecte a **saída 1** / **STANDARD** ao segundo **Direct Reply** e renomeie para **Standard**.

ANÁLISE PADRÃO

Análise do especialista:

`{{ agentAgentflow_0 }}`

O Condition Agent decide a rota. O Direct Reply encerra o caminho e exibe a análise técnica produzida pelo Incident Specialist.

25. Execute o Agentflow pelo chat local

No editor, abra o painel de **chat/teste** à direita e envie uma mensagem. O painel **Process Flow** mostra a trajetória executada.

25.1 Teste Review

O serviço de pagamentos está indisponível em produção após várias falhas. Existe a possibilidade de reiniciar o serviço, mas qualquer mudança exige aprovação humana. Ainda não há evidência suficiente para confirmar a causa raiz.

Esperado: Start -> Incident Specialist -> Condition Agent -> Review. A resposta começa com **REVISÃO HUMANA NECESSÁRIA**.

25.2 Teste Standard

Foi identificado um erro isolado em ambiente de desenvolvimento. A nova tentativa funcionou e não há impacto em produção. A recomendação é apenas coletar logs e acompanhar se ocorre novamente.

Esperado: Start -> Incident Specialist -> Condition Agent -> Standard.

26. Entenda os metadados da Prediction API

Campo	Função
text	Resposta final do Agentflow.
question	Mensagem que iniciou a execução.
chatId	Identificador da conversa.
chatMessageId	Identificador da mensagem.
executionId	Identificador da execução do Agentflow.
agentFlowExecutedData	Trajetoira dos nodes, inputs, outputs e status.
state	Flow State da execução; fica {} se não configurado.
previousNodeIds	IDs dos nodes anteriores. No Start, é [] porque não existe node anterior.

PARTE V - ORQUESTRAÇÃO HÍBRIDA N8N + FLOWISE

27. Exponha o Agentflow pela Prediction API

1. No **Flowise**, salve o **Agentflow**.
2. No canto superior direito do editor, clique no botão **</>**.
3. Abra a aba **cURL**.
4. Copie o endpoint completo da Prediction API. O **Flowise** local exibirá algo como **http://localhost:3001/api/v1/prediction/<FLOW_ID>**.
5. Se houver autorização configurada, preserve o header correspondente e nunca publique o token.

```
curl http://localhost:3001/api/v1/prediction/<FLOW_ID> \
-X POST \
-d '{"question":"Hey, how are you?"}' \
-H "Content-Type: application/json"
```

IMPORTANTE: esse cURL é para o host. No HTTP Request do n8n, que roda em outro container, substitua localhost:3001 por flowise:3000 e preserve o mesmo FLOW_ID.

28. Chame o Flowise a partir do n8n

1. No workflow principal, crie **HTTP Request** e renomeie para **Flowise Specialist Review**.
2. Conecte IF TRUE e Switch HIGH diretamente a esse node.
3. **Method** = POST.
4. URL = **http://flowise:3000/api/v1/prediction/<FLOW_ID>**.
5. Header **Content-Type** = **application/json**.
6. Body **Content Type** = **JSON**.

28.1 Body question

Envie dados objetivos e a análise primária já sanitizada. Assim o **Flowise** atua como segunda revisão, sem receber identificadores internos desnecessários.

```
{{ 'Faça uma segunda análise deste incidente. Destaque evidências, hipóteses, recomendação e incertezas.' +
  '\n\nDADOS OBJETIVOS:\n' + JSON.stringify($('Calculate Risk Score').first().json) +
  '\n\nANÁLISE PRIMÁRIA DO AGENTE N8N:\n' + $('Sanitizar Saída do Agente').first().json.output }}
```

1. Mantenha streaming = false, se o exemplo da sua versão incluir esse campo.

28.2 Teste de conectividade se houver erro

```
docker exec n8n node -e "fetch('http://flowise:3000/api/v1/ping').then(r => r.text()).then(console.log).catch(console.error)"
```

Se retornar pong, rede e nome do serviço estão corretos. Um erro "connection refused" usando localhost:3001 dentro do n8n indica que a URL está apontando para o container errado.

29. Atualize Prepare Human Review

O node final não deve analisar novamente. Ele consolida o pacote que será entregue a um humano ou a outro sistema.

1. Depois de Flowise Specialist Review, crie/atualize Edit Fields -> Prepare Human Review.
2. Mode = Manual Mapping e Include Other Input Fields = OFF.

Campo	Tipo	Origem / valor
eventId	String	{{ \$('Calculate Risk Score').first().json.id }}
component	String	{{ \$('Calculate Risk Score').first().json.component }}
environment	String	{{ \$('Calculate Risk Score').first().json.environment }}
riskScore	Number	{{ \$('Calculate Risk Score').first().json.riskScore }}
deterministicLevel	String	{{ \$('Calculate Risk Score').first().json.deterministicLevel }}
userId	Number	{{ \$('Calculate Risk Score').first().json.userId }}
content	String	{{ \$('Calculate Risk Score').first().json.content }}
agentAnalysis	String	{{ \$('Sanitizar Saída do Agente').first().json.output }}

specialistAnalysis	String	{{ \$('Flowise Specialist Review').first().json.text }}
specialistRoute	String	{{ \$('Flowise Specialist Review').first().json.agentFlowExecutedData.slice(-1)[0].nodeLabel }}
flowiseExecutionId	String	{{ \$('Flowise Specialist Review').first().json.executionId }}
flowiseChatId	String	{{ \$('Flowise Specialist Review').first().json.chatId }}
action	String	human_review_required
status	String	awaiting_human_review
reviewRequired	Boolean	true
reviewReason	String	high_risk_policy

Ou utilize o modo **JSON**:

```
{
  "eventId": {{ JSON.stringify($('Calculate Risk Score').first().json.id) }},
  "component": {{ JSON.stringify($('Calculate Risk Score').first().json.component) }},
  "environment": {{ JSON.stringify($('Calculate Risk Score').first().json.environment) }},
  "riskScore": {{ Number($('Calculate Risk Score').first().json.riskScore) }},
  "deterministicLevel": {{ JSON.stringify($('Calculate Risk Score').first().json.deterministicLevel) }},
  "userId": {{ Number($('Calculate Risk Score').first().json.userId) }},
  "content": {{ JSON.stringify($('Calculate Risk Score').first().json.content) }},
  "agentAnalysis": {{ JSON.stringify($('Sanitizar Saída do Agente').first().json.output) }},
  "specialistAnalysis": {{ JSON.stringify($('Flowise Specialist Review').first().json.text) }},
  "specialistRoute": {{ JSON.stringify($('Flowise Specialist Review').first().json.agentFlowExecutedData.slice(-1)[0].nodeLabel) }},
  "flowiseExecutionId": {{ JSON.stringify($('Flowise Specialist Review').first().json.executionId) }},
  "flowiseChatId": {{ JSON.stringify($('Flowise Specialist Review').first().json.chatId) }},
  "action": "human_review_required",
  "status": "awaiting_human_review",
  "reviewRequired": true,
  "reviewReason": "high_risk_policy"
}
```

Não copie system_fingerprint, contadores de tokens, state, previousNodeIds ou agentFlowExecutedData inteiro para o pacote humano. Esses dados são úteis para tracing, mas não para a decisão operacional. Extraia apenas o que interessa.

PRECEDÊNCIA: o n8n continua sendo a autoridade da política. Se riskScore/rota determinística exigiram revisão, um specialistRoute = Standard no Flowise não cancela a revisão humana. O Flowise funciona como segunda opinião especializada.

30. O que significa orquestração híbrida aqui?

Responsabilidade	n8n	Flowise
Entrada e integração	Webhook, transformação, HTTP e roteamento.	Recebe chamada pela Prediction API.
Decisão dinâmica	AI Agent do case principal.	Incident Specialist produz segunda análise.
Memória	Simple Memory por component.	Memória do Agentflow/conversa.
Controle	Filter, Guardrails, IF e Switch.	Condition Agent e limites do flow.
Ação final	Decide encaminhamento e registro.	Retorna análise e rota; não executa ação corretiva.

PARTE VI - VALIDAÇÃO, TROUBLESHOOTING E DESAFIO

31. Testes obrigatórios

Teste	O que mudar	O que observar
A	EVT-AG-001: impact 4, retries 3	riskScore 10; runbook chamado; IF TRUE; Flowise Review.
B	EVT-AG-002: mesmo payment-sync	Memória recupera informação anterior sem substituir o evento atual.
C	EVT-AG-003: inventory-sync	Nova sessão; não herda memória de payment-sync.
D	environment = development	Filter interrompe antes da IA.
E	impact 2, retries 0	LOW -> log only.
F	impact 3, retries 1	MEDIUM -> Wait -> follow up.
G	riskScore 8	Switch HIGH -> Flowise -> Prepare Human Review.
H	Texto com ação operacional proibida	Guardrails -> Fail Branch.
I	Saída normal sanitizada	Guardrails -> Pass Branch.
J	Prediction API	n8n chama flowise:3000 e recebe text/executionId.

32. Troubleshooting detalhado

Sintoma	Onde olhar	Correção
EVT-AG-002 vira EVT-AG-001	Normalize Event -> Output	Remover JSON fixo e mapear \$json.body.*.
Tool recebe null	Call n8n Workflow Tool -> Workflow Inputs	Mapear userId/component explicitamente.
Workflow is not active	Sub-workflow 03A/03B	Publish/Activate antes do AI Agent.
Memória não lembra	Simple Memory -> Key	Usar a mesma component e remover Pin Data.
Outra component herda histórico	Normalize Event / Simple Memory	Confirmar component dinâmica e Session Key correta.
Guardrails sempre FAIL	Guardrails -> Fail Branch -> checks	Ver check triggered e detectedSecrets; conferir sanitização.
Secret Keys marca functions.Call_*	guardrailsInput/detectedSecrets	Falso positivo; sanitizar identificadores internos.
riskScore some depois do Agent	IF / Switch	Referenciar Calculate Risk Score diretamente.
riskScore 8 não chega ao HIGH	IF threshold	Usar threshold didático 9 nesta prática.
Flowise não abre	docker compose logs -f flowise	Confirmar imagem 3.1.3 e volume; evitar 3.1.4 neste laboratório.
n8n -> Flowise connection refused	HTTP Request URL	Dentro do n8n use flowise:3000, não localhost:3001.
Teste de rede falha	docker network inspect agentic-lab	Confirmar n8n e flowise na mesma rede.
Condition Agent roteia errado	Input / Instructions / Scenarios	Selecionar NODE OUTPUTS correto pelo picker e conferir ordem 0/1.
Direct Reply mostra só rótulo	Direct Reply -> Message	Adicionar também o output do Incident Specialist pelo picker.
state = {}	Flowise Start / Flow State	Esperado quando Flow State não foi declarado.
previousNodeIds = []	agentFlowExecutedData[0]	Esperado no Start, que não possui node anterior.

33. Desafio para os alunos

Adapte a arquitetura para um processo auxiliar do seu projeto. O n8n deve permanecer como orquestrador principal e o Flowise deve atuar como especialista chamado por API em uma rota coerente.

33.1 Requisitos mínimos

- n8n e Flowise executando self-hosted no ambiente Docker do grupo;
- n8n como orquestrador principal;
- AI Agent com Chat Model;
- memória com Session Key justificável;
- duas tools autorizadas, com pelo menos uma consulta externa ou sub-workflow;
- um Guardrails nativo ou controle semântico equivalente;
- um gate determinístico fora do agente;
- pelo menos duas rotas de decisão;
- Flowise Agentflow V2 testado isoladamente;
- chamada n8n -> Flowise pela Prediction API;
- apenas ações seguras, reversíveis ou simuladas;

- objeto final estruturado para revisão ou encaminhamento.

33.2 Evidências que devem ser mostradas

Evidência	O que apresentar
Docker	docker compose ps e teste pong de n8n -> Flowise.
Canvas n8n	Fluxo principal, AI Agent, memória, tools, Guardrails e gates.
Tool use	Uma execução com tool e a sub-execução aberta.
Memória	Duas interações com mesma Session Key e uma com outra sessão.
Guardrail	Um PASS e um FAIL com motivo visível em checks.
Flowise	Agentflow salvo, Incident Specialist, Condition Agent e duas rotas.
Integração	HTTP Request do n8n chamando a Prediction API.
Rastreabilidade	executionId/chatId retornados pelo Flowise.
Revisão	Objeto final estruturado em Prepare Human Review.

CRITÉRIO: mais nodes não significam melhor arquitetura. Cada node, tool ou integração deve existir porque resolve uma responsabilidade clara.

34. Fechamento

A prática começa em um workflow controlado e adiciona autonomia apenas onde ela agrega valor. O AI Agent interpreta o contexto, usa memória e consulta tools. O Guardrails inspeciona a saída, enquanto riskScore, IF e Switch impõem políticas objetivas. O Flowise adiciona uma segunda análise especializada, acessada por API. O n8n permanece responsável pelo processo externo, pela política e pelo encaminhamento final.

WORKFLOW CONTROLA O PROCESSO -> AGENTE RESOLVE DECISÕES DINÂMICAS -> TOOLS AMPLIAM CAPACIDADES -> MEMÓRIA PRESERVA CONTEXTO -> GUARDRAILS LIMITAM AUTONOMIA -> FLOWISE ESPECIALIZA UMA ROTA

IDEIA FINAL: o objetivo de uma arquitetura agêntica não é maximizar autonomia. É colocar decisão dinâmica exatamente onde ela agrega valor e manter controle explícito onde risco, segurança e previsibilidade exigem.

35. Referências técnicas

- Docker Engine Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
- Docker Desktop Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>
- Docker Desktop WSL 2: <https://docs.docker.com/desktop/features/wsl/>
- n8n Docs: <https://docs.n8n.io/>
- n8n Advanced AI: <https://docs.n8n.io/advanced-ai/>
- Flowise - Get Started / Docker: <https://docs.flowiseai.com/getting-started>
- Flowise - Agentflow V2: <https://docs.flowiseai.com/using-flowise/agentflowv2>
- Flowise - Prediction API: <https://docs.flowiseai.com/using-flowise/prediction>
- Flowise - Issue 3.1.4 Docker startup: <https://github.com/FlowiseAI/Flowise/issues/6688>
- JSONPlaceholder: <https://jsonplaceholder.typicode.com/>

NOTA DE VERSÃO: interfaces de n8n e Flowise mudam com frequência. Este guia descreve o ambiente e os nomes observados/validados em 20/08/2026. Quando um rótulo visual mudar, preserve a responsabilidade arquitetural e procure a opção equivalente.

APÊNDICE A - ARQUIVOS DOCKER PARA O REPOSITÓRIO

A.1 Dockerfile.n8n

```
ARG N8N_BASE_IMAGE=docker.n8n.io/n8nio/n8n:latest
FROM ${N8N_BASE_IMAGE}
```

A.2 Dockerfile.flowise

```
FROM flowiseai/flowise:3.1.3
```

A.3 docker-compose.yml

Use o arquivo docker-compose.yml entregue junto deste guia. Ele contém os dois serviços, volumes nomeados e a rede externa agentic-lab. O conteúdo integral também aparece na Seção 4.4.

A.4 Sequência de inicialização

Linux	Windows PowerShell
cp .env.example .env	Copy-Item .env.example .env
docker network create agentic-lab	docker network create agentic-lab
docker compose up -d --build	docker compose up -d --build
docker compose ps	docker compose ps

```
docker exec n8n node -e "fetch('http://flowise:3000/api/v1/ping').then(r => r.text()).then(console.log).catch(console.error)"
```

Resultado esperado: pong. Depois abra n8n em <http://localhost:5678> e Flowise em <http://localhost:3001>.